`rewrite`: a programming language that learns its mathematics from rule files

Rainer Heintzmann
Friedrich-Schiller-Universität Jena, Germany
rainer.heintzmann@uni-jena.de

Abstract

`rewrite` is a small interpreted language for symbolic and numerical mathematics, written in the early 1990s and revived in 2026. Unlike most computer algebra systems, its kernel contains no mathematics: data types, operators, the laws of algebra, differentiation and integration rules are all defined in plain-text include files and applied by term rewriting. Rule sets ("algebras") are first-class objects that can be combined into sequences and fixpoint iterations. A `LIFT` statement transfers all rules of one set of operations to another, so that, for example, the rules of an abelian group written for $+$ also apply to $\cdot$ in a field. Numerical evaluation is interleaved with rewriting: as soon as all arguments of a function are numbers, an attached C function computes the value. We describe the design of the language, its revival on current 64-bit compilers including a regression test suite, and a case study in which part of the rule-based integrator Rubi was translated into `rewrite`. For this translation the language was extended by optional pattern parts with default values, commutative operators and lists of conditions. They are implemented by expanding each rule into its variants, which keeps the matcher unchanged but limits the approach to moderately sized rule sets.

1 Introduction

Computer algebra systems such as Mathematica [12] or SymPy [6] come with a large body of built-in mathematical knowledge. A user can add rules, but the core simplification and the calculus routines are part of the system and are mostly hidden from the user. Term rewriting systems such as Maude [4] take the opposite view: they provide a general rewriting engine and leave the mathematics to the user, but they are usually not aimed at numerical work.

`rewrite` sits between these positions. Its interpreter knows how to parse expressions with user-defined operators, how to match and replace terms, and how to call C functions for numerical values. Everything else is defined in include files: the sort `real`, the operators `+`, `*` and `^`, the group, ring and field laws, the differentiation rules, and numerical algorithms such as Simpson integration. The language was designed by the author between 1992 and 1996 on Unix and the Amiga, on top of a scanner and parser framework by Axel T. Schreiner. In 2026 it was ported to current compilers and extended with a test suite and a rule-based integration library.

The main contributions of this paper are:

- a description of a language in which mathematics is a library of rewrite rules, including the `LIFT` mechanism for transferring rules between algebraic structures;
- the interleaving of symbolic rewriting and numerical evaluation, which allows numerical algorithms to be written as rewrite rules;
- a case study translating rules of the rule-based integrator Rubi [7] into `rewrite`, with a discussion of what this translation requires from the matching engine.

2 Language design

2.1 Sorts, operators and numbers

A program first declares sorts (types) and operators. The following complete program defines real numbers with three operators and evaluates expressions:

```
SORT real                                   # a data type
NUM real                                    # numbers typed in have sort real
FUNC + : real,real >> real LA2 ATT plus     # left-associative, priority 2
FUNC * : real,real >> real LA3 ATT mal      # binds tighter than +
FUNC ^ : real,real >> real RA  ATT hoch     # right-associative
1+2*3                                       # prints 7
2^3^2                                       # prints 512
CONST x,y : real
x+y*2                                       # prints x + y * 2
```

The format words `LA1`–`LA5`, `RA` and `RU` define left-associative binary operators of five priorities, right-associative operators and right unary operators such as the factorial. `ATT name` attaches a C function from a table of user functions. It is called whenever all arguments of the function are numbers; otherwise the term stays symbolic. `OTHER` adds further meanings to an existing name for other argument sorts, so that `+` can mean addition of reals, complex numbers or vectors. `BRA` defines functions written with brackets, such as a scalar product $\langle a, b \rangle$.

Numbers are stored as C `double` values. Other sorts can carry arbitrary C data: the standard library defines complex numbers, real vectors and matrices, whose values are C arrays. Operations on whole vectors and matrices are executed by C functions, so they run at compiled speed.

2.2 Rules and algebras

Rules are collected in named rule sets, called *algebras*. A rule has a left side, a right side, an optional condition and an optional block of instructions that is executed before the replacement:

```
VAR A : real
ALG simp
SPEC    simp : A*1 = A              # SPEC starts a new rule set
ADDSPEC simp : A+0 = A              # ADDSPEC adds a rule
CONST u : real
REWR (u*1)+0 : simp                 # prints u
```

`VAR` declares meta variables that match any term of their sort. `NUM` declares meta variables that only match numbers, which makes rules like "evaluate if the argument is a number" easy to write. Conditions test whether a term occurs in another one (`IN`, `NIN`), compare terms in a total order (`LB`, used for sorting sums), or evaluate any expression with a boolean result.

Algebras are combined into lists. A list in parentheses is applied repeatedly until no rule changes the term any more. The standard rule list of the library is defined like this:

```
SPEC All : Std,Doit,(Linear,Compl,diff,GrRi),AGroup,integr,
           (Subst,GrRi),Std,Zus,unaerweg
```

With the library loaded, `REWR D(sin cos x,x) :  All` returns `- cos (cos (x ))* sin (x )`.

2.3 Transferring rules with LIFT

A distinctive feature of `rewrite` is the `LIFT` statement. It declares that all rules valid for one set of functions and constants also hold for another set. The standard library writes the laws of a group once and transfers them step by step:

```
LIFT (Ag+Ag,Ag-Ag,-Ag,0g) TO (Aag+Aag,Aag-Aag,-Aag,0ag)
LIFT (Aag+Aag,Aag-Aag,-Aag,0ag) TO (Ar+Ar,Ar-Ar,-Ar,0r)
LIFT (Aag+Aag,Aag-Aag,-Aag,0ag) TO (Ar*Ar,Ar/Ar,mi(Ar),1r)
```

The last line reuses the rules of an abelian group for multiplication, division, the multiplicative inverse `mi` and the neutral element 1 of a ring. In the same way, linearity is written once for a generic linear function and lifted to differentiation, integration, Taylor expansion and summation. This corresponds closely to the algebraic hierarchies of modern proof assistants [10], but here it acts directly on the rewrite rules.

2.4 Interleaving rewriting and numerics

After every single rewrite step, the interpreter evaluates all subterms whose arguments are numbers. Constants that have a value are replaced by their value when a term is copied. Together with the procedural part of the language (`FOR`, `WHILE`, `IF`, blocks with local constants, and user functions), this allows numerical algorithms to be written as rewrite rules. The library defines Simpson integration as a rule whose `DO` block computes the value:

```
ALG rechne
SPEC rechne : I(N,M,A,_x)=_myerg DO { _myerg=0;_zw1=0;_zw2=0  # Simpson
        FOR n=N+_step/2 TO M-_step STEP _step DO { _x=n;_zw1=_zw1+A
        _x=n+_step/2
        _zw2=_zw2+A }
        _x=M-_step/2
        _zw1=_zw1+A
        _x=N;_myerg=A+4*_zw1+2*_zw2
        _x=M;_myerg=(_myerg+A)*_step/2/3
        _x=dummi }
```

A second rule set first substitutes the integration variable by the internal variable `_x`. With `_step=0.01`, `REWR I(0,pi,sin(x),x) : numerik,rechne` returns 2. The same mechanism gives Newton's method and the search for minima and maxima. The symbolic part can prepare the numerical part: `newton` expects the derivative as an argument, which is computed symbolically by `D`.

The original application of the language was physics. The library contains physical constants, units and formulas, and an example program computes Bragg angles, structure factors and X-ray absorption of lithium niobate crystals from tabulated atomic data.

3 Case study: rule-based integration

Differentiation is easy to express with rewrite rules, because every rule takes the term apart and the product and chain rules always apply. Integration is harder. Complete decision procedures exist for elementary functions [8, 2], but they are not pattern based. Rubi [7, 9] showed that an extensive system of carefully ordered rules with conditions can integrate a very large class of functions. Its rule files contain about 8500 rule definitions in Mathematica syntax, published under the MIT license. We translated a closed subset of them to test how far the mechanisms of `rewrite` carry.

3.1 Translation scheme

Rubi's patterns rely on features of the Mathematica pattern matcher that the original `rewrite` did not have. A typical rule is

```
Int[(a_. + b_.*x_)^m_, x_Symbol] := (a + b*x)^(m + 1)/(b*(m + 1)) /;
    FreeQ[{a, b, m}, x] && NeQ[m, -1]
```

where `a_.` matches a missing summand with default 0 and `b_.` a missing factor with default 1. The pattern also matches $x\,b + a$, because Mathematica matches sums and products in any order, and the condition combines several tests. We added three corresponding features to the language.

Optional parts. `OPT a,b` declares meta variables that may be missing. `DEFAULT + = 0` and `DEFAULT * = 1` give the value of a missing operand, and `DEFAULT RIGHT ^ = 1` allows only the exponent of a power to be missing.

Commutative operators. `COMM +,*` declares that patterns with these operators also match with swapped operands.

Condition lists. A rule can have several conditions separated by commas, all of which must hold. `NIN(x,a,b,m)` tests that x occurs in none of a, b, m, which is Rubi's `FreeQ`.

All three are implemented when a rule is defined: the rule is expanded into all its variants, with the default values of missing variables substituted into the right-hand side, the condition and the `DO` block. The matcher itself is unchanged. With these features the rule above translates almost one to one:

```
ADDSPEC IntRubi : IF NIN(Xv,Qa,Qb,Pm), (Pm != -1) Int((Qa+Qb*Xv)^Pm,Xv) =
                  (Qa+Qb*Xv)^(Pm+1)/(Qb*(Pm+1))
```

Here `Pm` is a required and `Qa`, `Qb` are optional meta variables. One rule now covers $(a+bx)^m$, $(bx+a)^m$, $(a+x)^m$, $(bx)^m$ and x^m, with numeric or symbolic coefficients.

Rubi also relies on Mathematica's normal form of expressions. A small rule set produces a similar form before the Rubi rules are applied: $x - y$ becomes $x + (-1)\,y$, x/y becomes $x\,y^{-1}$, $\sqrt{x}$ becomes $x^{0.5}$, numeric factors move to the front and products with sums are multiplied out. Rubi expresses $\cos z$ as $\sin(z + \pi/2)$; the cosine rules were obtained by working out this shift. The complete integration is the rule list `(IntNorm,IntRubi),IntOut,All`, and the original Mathematica form of each rule is kept as a comment in the rule files.

Helper functions. Many Rubi rules use helper functions in their conditions and right-hand sides, such as `ExpandIntegrand`, `PolyQ`, `IntegerQ` or `Subst`. They cannot be ordinary `rewrite` functions, which are only evaluated when all arguments are numbers. We therefore added symbolic C functions: they receive the argument terms and return a new term, or nothing if they do not apply. Predicates return true or false and can be combined in conditions, for example `IF (polyq(u,x) && (expon(u,x) == 2))`. The helpers implemented so far cover the most frequent ones, together with missing elementary functions such as csc, arctanh or the polylogarithm. Some are simplified versions: `expand` multiplies out and collects like terms but does no partial fractions, and `simp` returns its argument. Evaluation order needed care: Rubi's `Subst[Int[u],x,v]` integrates before substituting, so our `subst` waits while its argument still contains an unevaluated integral. A rule of the form $\int u \to \int \mathrm{ExpandIntegrand}(u)$ loops if the expansion makes no progress, so `expandintegrand` then stays unevaluated.

3.2 Results

The subset consists of 13 Rubi rules from sections 9.1 (integrand simplification), 1.1.1.1 ($(a+bx)^m$), 1.1.1.2 ($(a+bx)^m(c+dx)^n$), 2.1 and 2.3 (exponentials), and 4.1.1.1 and 4.1.10 (sine), which became 15 `rewrite` rules. It integrates sums of powers and products of powers of linear expressions, possibly multiplied by one exponential, sine or cosine of a linear argument, with numeric or symbolic coefficients. Table 1 shows examples. The test suite checks 31 integrals by differentiating each result again and comparing the derivative numerically with the integrand.

When they are defined, the 15 rules expand into 1676 variants. Most rules have between 2 and 9 variants, but the three rules for a power of a linear expression times an exponential, sine or cosine have five optional parameters and two commutative products, and expand into 324 variants each; the rule for products of two powers has 647 variants. Loading the integration rules takes about 0.1 s.

Of Rubi's 8492 rule definitions, 72% use only functions that are now available in `rewrite`. With a translator that resolves Mathematica's local definitions (`With`, `Module`), 77% would be translatable.

Integrand	Result of `REWR Int(f,x) : Integrate`
`1/(2*x+3)`	`0.5 * ln (3 + 2 * x )`
`5/(x+1)^2`	`- 5 / (1 + x )`
`sqrt(x)`	`0.666667 * x ^ 1.5`
`x*exp(2*x)`	`0.5 * exp (2 * x )* x - 0.25 * exp (2 * x )`
`x^2*cos(x)`	`2 * cos (x )* x + x ^ 2 * sin (x )- 2 * sin (x )`
`sin(a*x+b)`	`- cos (a * x + b )/ a`
`x*exp(a*x)`	`exp (a * x )* x / a - exp (a * x )/ a ^ 2`
`1/(a+b*x)`	`ln (b * x + a )/ b`
`sqrt(x)*sin(x)`	`0.5 * Int (cos (x )/ x ^ 0.5 ,x )- x ^ 0.5 * cos (x )`

Table 1: Indefinite integrals computed with the translated Rubi subset. In the last example the reduction stops at an integral that would need Fresnel functions, which are not part of the subset.

During this work, the translation also served as a test of the library itself. It revealed two incorrect simplification rules in the standard algebra, which turned $\ln(x+y)$ into $\ln x \cdot \ln y$ and a^{b^c} into a^{bc}, and two incorrect derivatives (of arcsin and arccos). They have been corrected.

4 Implementation and revival

The interpreter consists of about 8100 lines of C and a grammar of about 480 lines for the scanner and parser generators flex and bison. Terms are binary trees of nodes taken from memory pools; symbols carry their sort, value and the links used by `LIFT`. The rules of an algebra are merged into a single tree, in which rules with a common left-hand-side structure share nodes and alternatives branch off where they differ. This tree is matched against every subterm, innermost first.

The program was written for 32-bit Unix and the Amiga and no longer worked with current tools. The port to 64-bit Linux and Windows (MSYS2) required only small changes to the C code itself. The main problems were changed semantics in the scanner generator, the release of static memory that old C libraries tolerated, and errors in the handling of declarations inside loops and included files. A regression test suite with 25 test programs now covers the core language, the library, the new pattern features and helper functions, the integration rules and the original example programs. Memory leaks were removed with an instrumented build that counts allocations per source line, so that repeated calculations run in constant memory. All parser structures are now accessed through named macros instead of chains like `np->r->r->l`;

for most source files the compiled code stayed byte-identical, which confirmed that this cleanup did not change the behaviour.

Performance is adequate for interactive use. Loading the complete standard library and computing a derivative or an integral takes well below 0.1 s on a current laptop. A scalar loop written in the language itself needs about 0.25 μs per iteration (measured for 10^7 iterations of `s = s + k*k`). We estimate this to be about two orders of magnitude slower than an equivalent compiled loop. Operations on whole vectors and matrices run in C.

5 Discussion

Strengths. The main strength of `rewrite` is that all mathematics is visible and changeable. Correcting the derivative of arcsin meant editing one line of a text file, not the interpreter. `LIFT` allows algebraic structure to be expressed once and reused, and `NUM` variables together with conditions give precise control over when a rule fires. The interleaving of rewriting and numerical evaluation makes it natural to derive formulas symbolically and evaluate them numerically in the same program, for example derivatives for Newton's method.

Limitations. Optional parts and commutative operators are implemented by expanding rules into variants, not inside the matcher. The number of variants grows exponentially with the number of optional parameters and commutative products, so the approach suits rule sets of moderate size but not all of Rubi's rules. Associativity is only handled by normalization rules. Nothing checks that a rule set terminates or is confluent [1], and the order of rules matters: during the translation, two normalization rules defined in the wrong order made the interpreter alternate forever between $5(-a)$ and $-1 \cdot (5a)$. Numeric conditions such as $m \neq -1$ do not hold for symbolic m, which is more conservative than Rubi. Error messages are minimal.

Related work. Rule-based simplification is at the core of Mathematica [12] and of Rubi [7]. General rewriting logic is provided by Maude [4]. More recently, equality saturation with e-graphs, as in egg [11] and Metatheory.jl [3], avoids the dependence on rule order by representing many equivalent terms at once. Proof assistants organize mathematical knowledge in algebraic hierarchies and simplification rule sets [10]. `rewrite` anticipated several of these ideas in a small and self-contained system.

Outlook. Natural next steps are moving optional arguments and commutative matching into the matcher, which would avoid the expansion into variants; an automatic translator from Rubi's Mathematica syntax for the rules whose helper functions are now available (about 72% of them); the remaining helper functions such as partial fractions; and code generation that turns symbolically derived expressions into compiled numerical code.

6 Conclusion

`rewrite` shows that a small interpreter can host a complete computer algebra library when the mathematics is expressed as rewrite rules and rule sets can be combined and lifted between algebraic structures. The translation of part of Rubi demonstrates that established rule-based algorithms can be imported, and it clarifies which matching features such an import requires.

Availability

The source code, the standard library, the integration rules and the test suite are available at `https://github.com/RainerHeintzmann/rewrite` under the MIT license [5]. The translated

integration rules are derived from Rubi [9] and keep its MIT license notice.

Acknowledgements

The scanner and parser framework of the original program is based on code by Axel T. Schreiner. The 2026 port, the test suite, the memory leak analysis and the translation of the integration rules were carried out with the assistance of an AI coding assistant (Claude, Anthropic).

References

[1] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.

[2] Manuel Bronstein. *Symbolic Integration I: Transcendental Functions*. Springer, 2 edition, 2005.

[3] Alessandro Cheli. Metatheory.jl: Fast and elegant algebraic computation in Julia with extensible equality saturation. *Journal of Open Source Software*, 6(59):3078, 2021.

[4] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, and Carolyn Talcott. *All About Maude – A High-Performance Logical Framework*, volume 4350 of *Lecture Notes in Computer Science*. Springer, 2007.

[5] Rainer Heintzmann. rewrite – a computer language that first has to learn mathematics from include files. `https://github.com/RainerHeintzmann/rewrite`.

[6] Aaron Meurer et al. SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3:e103, 2017.

[7] Albert Rich, Patrick Scheibe, and Nasser M. Abbasi. Rule-based integration: An extensive system of symbolic integration rules. *Journal of Open Source Software*, 3(32):1073, 2018.

[8] Robert H. Risch. The problem of integration in finite terms. *Transactions of the American Mathematical Society*, 139:167–189, 1969.

[9] Rule-Based-Integration Organization. Rubi – rule-based integrator, version 4.17.3. `https://github.com/RuleBasedIntegration/Rubi`. MIT license.

[10] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020.

[11] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, 5(POPL), 2021.

[12] Wolfram Research, Inc. Mathematica. Champaign, IL, `https://www.wolfram.com/mathematica`.